

Recommendation Engine API Documentation

Base URL

```
http://localhost:8000
```

Authentication

Currently no authentication required (add for production)

1. System Endpoints

Health Check

Check if the service is running and ML features status.

Request:

```
bash

GET /health
```

Response:

```
json

{
  "status": "healthy",
  "service": "rec-engine",
  "ml_enabled": true
}
```

Usage: Monitor service availability, check ML status

API Documentation

```
bash
```

GET /docs # Interactive Swagger UI

GET /openapi.json # OpenAPI schema

2. User Management

Create User

Register a new user in the system.

Request:

```
bash
```

```
POST /users/
```

```
Content-Type: application/json
```

```
{  
  "username": "john_doe",  
  "email": "john@example.com"  
}
```

Response:

```
json
```

```
{  
  "id": "550e8400-e29b-41d4-a716-446655440000",  
  "username": "john_doe",  
  "email": "john@example.com",  
  "created_at": "2024-01-07T10:30:00"  
}
```

Use Case: User registration, customer onboarding

Get User

Retrieve user details by ID.

Request:

```
bash
```

```
GET /users/{user_id}
```

Example:

```
bash
```

```
curl http://localhost:8000/users/550e8400-e29b-41d4-a716-446655440000
```

List Users

Get all users with pagination.

Request:

```
bash
```

```
GET /users/?skip=0&limit=100
```

Get User Preferences

Analyze user behavior and preferences based on interactions.

Request:

```
bash
```

```
GET /users/{user_id}/preferences
```

Response:

```
json
```

```
{
  "user": {
    "id": "550e8400...",
    "username": "john_doe",
    "email": "john@example.com"
  },
  "preferences": {
    "total_interactions": 45,
    "favorite_categories": [
      {"category": "Mobiles", "count": 15},
      {"category": "Electronics", "count": 10}
    ],
    "average_rating": 4.2
  }
}
```

Use Case: Personalization, user analytics, targeted marketing

3. Item Management

Create Item

Add a new product to the catalog.

Request:

```
bash
```

POST /items/

Content-Type: application/json

```
{  
  "title": "iPhone 15 Pro",  
  "description": "Latest Apple smartphone",  
  "category": "Mobiles",  
  "brand": "Apple",  
  "price": 999.99,  
  "in_stock": true,  
  "product_status": "active",  
  "category_name": "Mobiles",  
  "product_rating": 4.5,  
  "discount": 10.0  
}
```

Response: Created item with generated ID

Use Case: Product catalog management

Get Item

Retrieve specific item details.

Request:

bash

GET /items/{item_id}

List Items

Browse items with filters and pagination.

Request:

bash

GET /items/?category=Mobiles&in_stock_only=true&skip=0&limit=20

Query Parameters:

- `category`: Filter by category
- `in_stock_only`: Show only available items
- `skip`: Offset for pagination
- `limit`: Number of items to return

Use Case: Product browsing, catalog display

4. Search APIs

Enhanced Search

Advanced search with facets and filters.

Request:

```
bash

POST /search/enhanced/
Content-Type: application/json

{
  "query": "smartphone",
  "category": "Mobiles",
  "min_price": 100,
  "max_price": 1000,
  "min_rating": 4.0,
  "in_stock_only": true,
  "sort_by": "price_asc",
  "limit": 20,
  "offset": 0
}
```

Response:

```
json
```

```
{
  "items": [...],
  "total": 45,
  "query": "smartphone",
  "filters_applied": {
    "category": "Mobiles",
    "price_range": [100, 1000]
  },
  "suggestions": ["smartphone case", "smartphone accessories"],
  "facets": {
    "categories": [
      {"value": "Mobiles", "count": 30},
      {"value": "Accessories", "count": 15}
    ],
    "brands": [
      {"value": "Apple", "count": 10},
      {"value": "Samsung", "count": 20}
    ]
  }
}
```

Sort Options:

- `relevance`: Default, by match score
- `price_asc`: Lowest price first
- `price_desc`: Highest price first
- `rating`: Highest rated first
- `popularity`: Most popular first

Use Case: Product search, filtering, faceted navigation

Semantic Search (ML)

AI-powered search that understands meaning and context.

Request:

```
bash
```

```
POST /search/semantic/  
Content-Type: application/json
```

```
{  
  "query": "wireless music player",  
  "limit": 10  
}
```

Features:

- Understands synonyms (phone → smartphone → mobile)
- Finds related concepts (music player → headphones, speakers)
- Context-aware matching

Use Case: Intelligent search when exact keywords don't match

NLP-Enhanced Search (ML)

Natural language search with intent extraction.

Request:

```
bash  
  
POST /search/nlp-enhanced/  
Content-Type: application/json  
  
{  
  "query": "show me cheap phones under 500 dollars with good camera",  
  "limit": 10  
}
```

Features:

- Extracts price filters from text
- Understands product attributes
- Expands queries with synonyms
- Detects user intent

Use Case: Voice search, conversational commerce

5. Recommendation APIs

Enhanced Recommendations

Multi-algorithm recommendations with intelligent selection.

Request:

```
bash

POST /recommend/enhanced/
Content-Type: application/json

{
  "user_id": "550e8400-e29b-41d4-a716-446655440000",
  "algorithm": "auto",
  "limit": 20,
  "exclude_items": ["item_id_1", "item_id_2"]
}
```

Algorithms:

- `auto`: System selects best algorithm
- `collaborative`: Based on similar users
- `content`: Based on item features
- `hybrid`: Combination of methods
- `deep_learning`: Neural network predictions

Response:

```
json
```

```
{
  "items": [
    {
      "item": {...},
      "score": 0.95,
      "reason": "Based on your purchase history",
      "algorithm_used": "collaborative"
    }
  ],
  "user_id": "550e8400...",
  "algorithm_used": "hybrid",
  "total": 20
}
```

Use Case: Homepage recommendations, personalized emails

Deep Learning Recommendations (ML)

Neural network-based personalized recommendations.

Request:

```
bash

POST /recommend/deep-learning/
Content-Type: application/json

{
  "user_id": "550e8400-e29b-41d4-a716-446655440000",
  "limit": 10
}
```

Use Case: High-accuracy personalization for engaged users

6. Similar Items

Get Similar Items (Basic)

Find items in the same category.

Request:

```
bash
```

```
GET /items/{item_id}/similar?limit=10
```

Get Similar Items (ML-Enhanced)

AI-powered similarity using multiple factors.

Request:

```
bash
```

```
GET /items/{item_id}/similar/enhanced/?limit=10
```

Response:

```
json
{
  "items": [
    {
      "item": {...},
      "score": 0.92,
      "reason": "Similar features and price range",
      "algorithm_used": "content"
    }
  ],
  "item_id": "original_item_id",
  "algorithm_used": "content",
  "total": 10
}
```

Similarity Factors:

- Description embeddings
- Category matching
- Price range ($\pm 30\%$)
- Brand alignment
- User co-views

Use Case: "You might also like", "Similar products"

7. User Interactions

Record Interaction

Track user behavior for recommendations.

Request:

```
bash

POST /interactions/
Content-Type: application/json

{
  "user_id": "550e8400-e29b-41d4-a716-446655440000",
  "item_id": "item_123",
  "interaction_type": "purchase",
  "rating": 5.0
}
```

Interaction Types:

- `view`: User viewed item
- `click`: User clicked on item
- `purchase`: User bought item
- `like`: User favorited item

Rating: 1.0 to 5.0 (optional)

Use Case: Behavior tracking, implicit feedback

8. Analytics

Trending Items

Get AI-predicted trending products.

Request:

```
bash
```

```
GET /analytics/trending-items/?window_days=7&limit=20
```

Response:

```
json
{
  "trending_items": [
    {
      "item": {...},
      "trend_score": 2.5,
      "interaction_count": 150
    }
  ]
}
```

Use Case: "Trending now" sections, inventory planning

Popular Items

Get most popular products.

Request:

```
bash
```

```
GET /popular/?limit=20
```

Use Case: Bestsellers, default recommendations

9. ML Management

Check ML Status

Verify which ML features are available.

Request:

```
bash
```

```
GET /ml/status/
```

Response:

```
json
{
  "ml_enabled": true,
  "engines": {
    "ml_engine": true,
    "nlp_engine": true,
    "image_engine": true,
    "predictive_engine": true
  },
  "models_directory": true,
  "endpoints_available": [
    "/search/semantic/",
    "/search/nlp-enhanced/",
    "/recommend/deep-learning/",
    "/analytics/trending-items/",
    "/ml/train-models/"
  ]
}
```

Train Models

Trigger ML model training (requires 100+ interactions).

Request:

```
bash
POST /ml/train-models/
```

Response:

```
json
```

```
{  
  "message": "Model training started in background",  
  "current_interactions": 500  
}
```

Complete Workflow Examples

1. New User Journey

```
python  
  
import requests  
  
API_URL = "http://localhost:8000"  
  
# 1. Create user  
user = requests.post(f"{API_URL}/users/",  
    json={"username": "alice", "email": "alice@example.com"}).json()  
  
# 2. User searches for products  
search_results = requests.post(f"{API_URL}/search/enhanced/",  
    json={"query": "laptop", "max_price": 1500}).json()  
  
# 3. User views a product  
item_id = search_results["items"][0]["id"]  
requests.post(f"{API_URL}/interactions/",  
    json={"user_id": user["id"], "item_id": item_id,  
        "interaction_type": "view", "rating": 4.0})  
  
# 4. Get recommendations  
recommendations = requests.post(f"{API_URL}/recommend/enhanced/",  
    json={"user_id": user["id"], "limit": 10}).json()  
  
# 5. Get similar items  
similar = requests.get(  
    f"{API_URL}/items/{item_id}/similar/enhanced/?limit=5").json()
```

2. Search and Filter Flow

```
python
```

Semantic search for concept

```
results = requests.post(f'{API_URL}/search/semantic/',  
    json={"query": "gaming computer"}).json()
```

Natural language search

```
results = requests.post(f'{API_URL}/search/nlp-enhanced/',  
    json={"query": "find me a phone under 30000 with good battery"}).json()
```

Faceted search with filters

```
results = requests.post(f'{API_URL}/search/enhanced/',  
    json={  
        "query": "phone",  
        "category": "Mobiles",  
        "min_price": 10000,  
        "max_price": 30000,  
        "min_rating": 4.0,  
        "sort_by": "price_asc"  
    }).json()
```

3. Analytics Dashboard

python

Get trending items

```
trending = requests.get(f'{API_URL}/analytics/trending-items/').json()
```

Get popular items

```
popular = requests.get(f'{API_URL}/popular/?limit=10').json()
```

Check user preferences

```
prefs = requests.get(f'{API_URL}/users/{user_id}/preferences').json()
```

Check ML system health

```
ml_status = requests.get(f'{API_URL}/ml/status/').json()
```

Error Handling

All endpoints return standard HTTP status codes:

- **200**: Success
- **400**: Bad request (invalid input)

- `404`: Resource not found
- `500`: Server error
- `503`: Service unavailable (ML features)

Error Response Format:

```
json
{
  "detail": "Error message here"
}
```

Performance Tips

1. **Caching**: Results are cached for 5 minutes
2. **Pagination**: Use `skip` and `limit` for large datasets
3. **Batch Operations**: Create multiple items in sequence
4. **Async Processing**: Recommendations process in background
5. **Index Building**: Happens automatically on startup

Next Steps

1. Add authentication/authorization
2. Implement rate limiting
3. Add webhook support for real-time updates
4. Enable A/B testing for algorithms
5. Add more detailed analytics endpoints